Data Structures and Algorithms -3 (25CS2103E)

HashGuard – Plagiarism Detection Platform

Team Members

Sohith Sai Veldi - 2520030389

Lekkala Pranavi - 2520030050

Under the Esteemed Guidance of

Mrs. J. Yamini Devi

Assistant Professor, Department of CSE

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

HashGuard is a Java-based, browser-accessible plagiarism detection application that compares text from two documents and identifies matching passages.

The detection engine uses the Rabin–Karp string-matching method with a polynomial rolling hash. Text is normalized, hashes are calculated for fixed-size windows, and each hash match is verified against the actual text to reject collisions.

The application displays matching passages and document similarity estimates through a web interface. It demonstrates how hashing, string processing, and efficient search can be applied to a practical document-comparison task.

Keywords: Plagiarism detection, Rabin–Karp, rolling hash, string matching, Java, text normalization.

Introduction

Background

Plagiarism detection helps users identify repeated or copied text in assignments, reports, and other documents. Manual checking becomes difficult as document length and the number of comparisons increase.

Project overview

HashGuard provides a web interface for account access, document scanning, comparison results, and highlighted matching passages. The application is implemented in Java and uses the JDK's built-in HTTP server.

DSA connection

The core task is finding common text windows efficiently. Rabin–Karp uses hash values to shortlist candidate matches, while direct string comparison confirms that a match is genuine.

Scope

The current implementation focuses on exact text matches after normalization. It does not reliably identify paraphrased meaning or establish intent to plagiarize.

Literature Survey

Naïve string matching

Checks a pattern at each possible text position. It is easy to understand but can repeat many character comparisons.

Rabin–Karp algorithm

Uses a hash of the pattern and rolling hashes of text windows. Hash matches are verified with direct comparison; this is the main algorithm used by HashGuard.

Rolling polynomial hash

Updates a window hash in constant time per shift, avoiding recalculation of every character contribution.

Text normalization

Lowercasing and standardizing punctuation/spacing can make equivalent text easier to compare, but aggressive normalization may remove meaningful distinctions.

Design decision

HashGuard indexes hashes for one document and checks windows from the other document. The current code does not implement KMP, Z-algorithm, suffix arrays, or semantic language models.

Problem Statement and Objectives

Problem statement

Manual comparison of lengthy documents is time-consuming and can overlook repeated passages. A practical tool is needed to automate text comparison and present likely overlaps clearly.

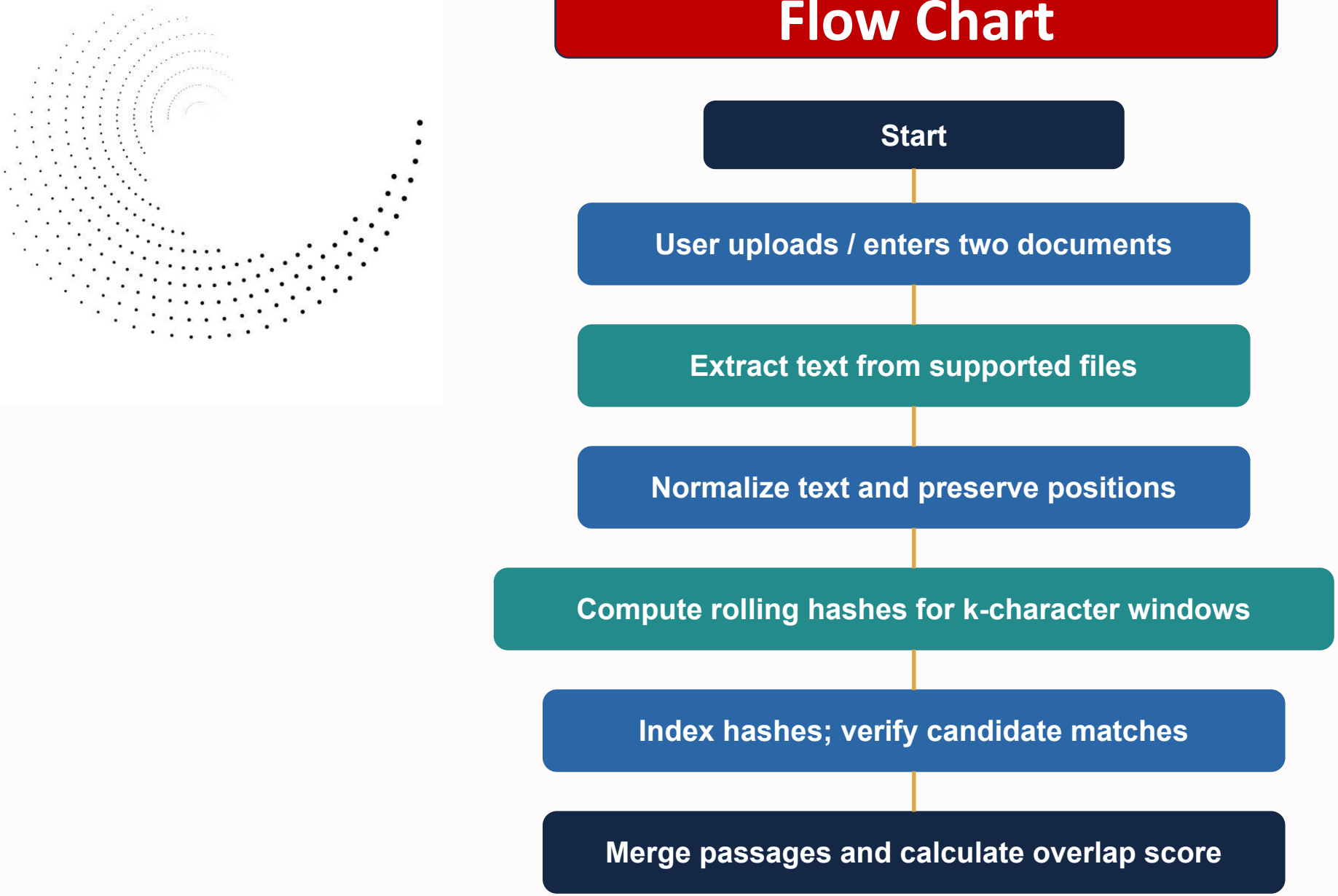
Objectives

- Accept supported text documents or pasted text.
- Normalize text before comparison.
- Use Rabin–Karp rolling hashes to find candidate matching windows.
- Verify hash hits character-by-character to handle collisions.
- Merge adjacent matches into passages and highlight them.
- Display document-level similarity estimates in a browser interface.

Success criteria

The application should start locally, accept supported input, find exact repeated text after normalization, and present results in an understandable format.

Flow Chart


Start
User uploads / enters two documents
Extract text from supported files
Normalize text and preserve positions
Compute rolling hashes for k-character windows
Index hashes; verify candidate matches
Merge passages and calculate overlap score

Results

Implemented features

- Java HTTP server and browser interface.
- Sign-up / login screens and dashboard flow.
- Text comparison using rolling polynomial hashes.
- Collision verification and matching-passage grouping.
- Similarity indicators and highlighted matches.

Input formats

TXT and DOCX text are read by the Java server. PDF text extraction is performed in the browser using PDF.js and requires that library to load.

Observed run status

The Java source compiled successfully. A later launch reported that the address was already in use, which indicates another process was already bound to the configured port; this is not a compilation error.

Important interpretation

The score is an overlap estimate based on normalized character coverage. It is not a universal plagiarism verdict. Add screenshots of the running page and real comparison results before presenting.

Hardware and Software requirements

Hardware

- Laptop or desktop computer.
- Modern multi-core processor recommended.
- Sufficient RAM for the operating system, browser, and Java runtime.
- Storage for source files and sample documents.

Software

- Windows, macOS, or Linux.
- JDK 17 or newer.
- Visual Studio Code or another Java editor (optional).
- Modern browser such as Chrome or Edge.
- JDK built-in com.sun.net.httpserver HTTP server; no Maven or Gradle required.

Libraries / data

- PDF.js is used in the browser for PDF text extraction and requires internet access unless bundled locally.
- User records are stored in data/users.tsv using salted PBKDF2 password hashes.
- Scan history is kept in memory and resets when the server restarts.

Conclusion

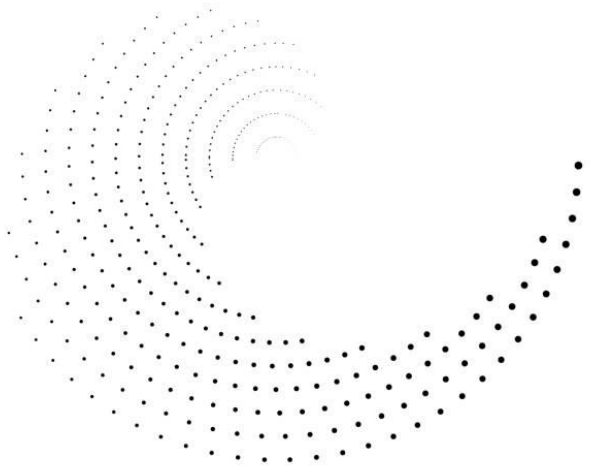
HashGuard demonstrates a practical application of Data Structures and Algorithms in document comparison. Its Java application combines text normalization, polynomial rolling hashes, hash indexing, collision verification, and grouping of matching windows.

The web interface presents matches in a way that is easier to inspect than manual comparison alone. The project also demonstrates the relationship between algorithm design and a usable software application.

The current system is strongest at detecting exact or near-exact passages after normalization. It should be treated as an assistance tool: its similarity score does not by itself prove plagiarism, and heavily reworded text may be missed.

Future Scope

- Add KMP and Z-algorithm implementations for educational comparison of string-matching methods.
- Improve handling of short passages and configurable matching-window sizes.
- Support more document formats and improve extraction from complex PDFs.
- Add downloadable reports with matched passages and document statistics.
- Introduce semantic similarity methods to detect paraphrased content.
- Move account management and persistent scan history to a secure backend/database if multi-user deployment is required.
- Bundle external libraries locally and add more systematic performance and correctness tests.



Thank you